Assignment 6 — AWS Deployment

Name: Umair Khan
Course: DevOps | Tutedude
Date: October 11, 2026
GitHub Repository: https://github.com/Umair-bey/tutedude-assignments
AWS Region: ap-south-1 (Mumbai)

1. Introduction

The objective of this assignment was to deploy a full-stack application — a **Flask backend** and an **Express frontend** — on Amazon Web Services using three different deployment architectures:

1. **Task 1:** Both applications on a **single Amazon EC2** instance
2. **Task 2:** Applications deployed on **separate EC2 instances** (backend EC2 + frontend EC2) communicating over the network
3. **Task 3:** Applications **dockerized** and deployed using **AWS ECR**, **ECS (Fargate)**, and a custom **VPC**

The purpose was to demonstrate practical experience with EC2 provisioning, networking, security groups, Linux server administration, Docker containerization, image registries, container orchestration, and network architecture (VPC, subnets, IGW, route tables).

2. Project Structure

Assignment-6/

```
├── backend/
│   ├── app.py              # Flask REST API (port 5000)
│   ├── requirements.txt    # flask, flask-cors
│   └── Dockerfile
├── frontend/
│   ├── app.js              # Express server (port 3000)
│   ├── package.json
│   ├── views/form.ejs      # HTML template
│   └── Dockerfile
└── README.md
```

Backend (Flask) provides:

- `GET /api/items` — list items
- `POST /api/items` — add an item
- `GET /health` — health check

Frontend (Express) renders a form that displays items fetched from the Flask backend using the `BACKEND_URL` environment variable.

3. Task 1 — Deployment on a Single EC2 Instance

3.1 EC2 Instance Details

Field	Value
Instance Name	`tutede-single-ec2`
Instance ID	`i-0fd33e246d93e23c4`
Region	ap-south-1 (Mumbai)
AMI	Ubuntu Server 24.04 LTS
Instance Type	t3.micro (Free Tier)
Key Pair	`tutedude-aws-key-mumbai`
Security Group	`tutede-aws-sg`

3.2 Security Group Rules

Type	Port	Source	Purpose
SSH	22	My IP	Remote access

Custom TCP	3000	0.0.0.0/0	Express frontend
Custom TCP	5000	0.0.0.0/0	Flask backend

3.3 Deployment Commands

bash

```
# SSH into the instance
ssh -i "tutedude-aws-key-mumbai.pem" ubuntu@<EC2-IP>

# Install dependencies
sudo apt update && sudo apt install -y python3 python3-pip python3-venv nodejs npm git

# Clone the repository
git clone https://github.com/Umair-bey/tutedude-assignments.git
cd tutede-assignments/Assignment-6

# Deploy backend
cd backend
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
nohup python app.py > backend.log 2>&1 &

# Deploy frontend
```

cd ../frontend

npm install

BACKEND_URL=http://localhost:5000 nohup node app.js > frontend.log 2>&1 &

3.4 Screenshots

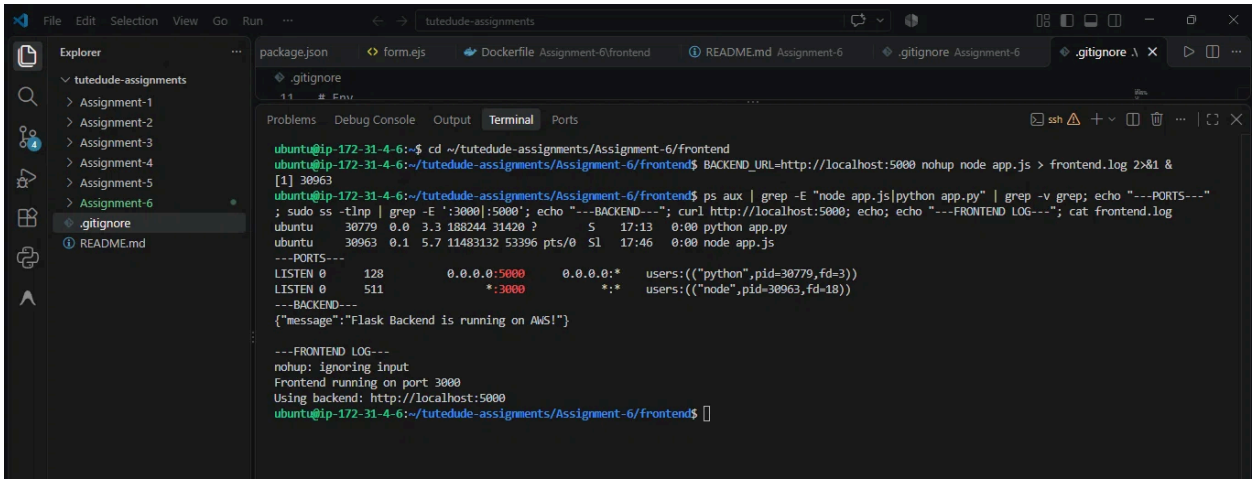


Figure 1: Flask + Express both running on the single EC2 (ports 3000 and 5000 open).

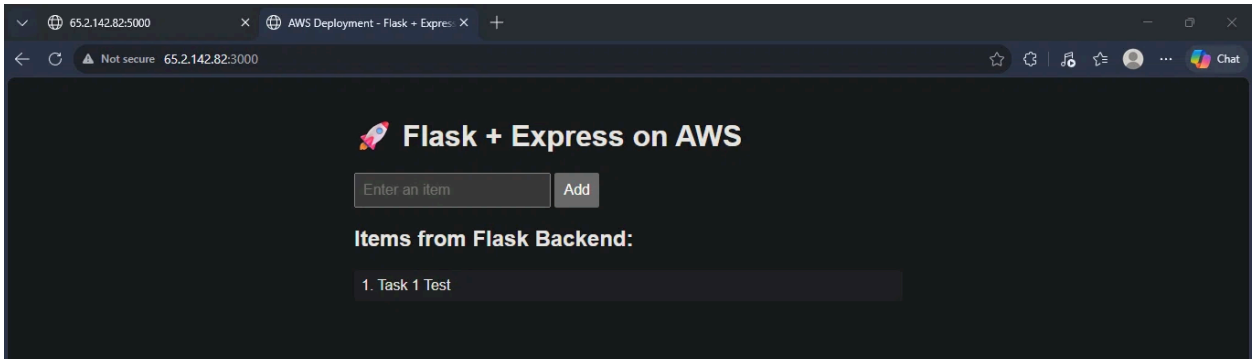


Figure 2: Live application accessible at http://65.2.142.82:3000 with an item added.

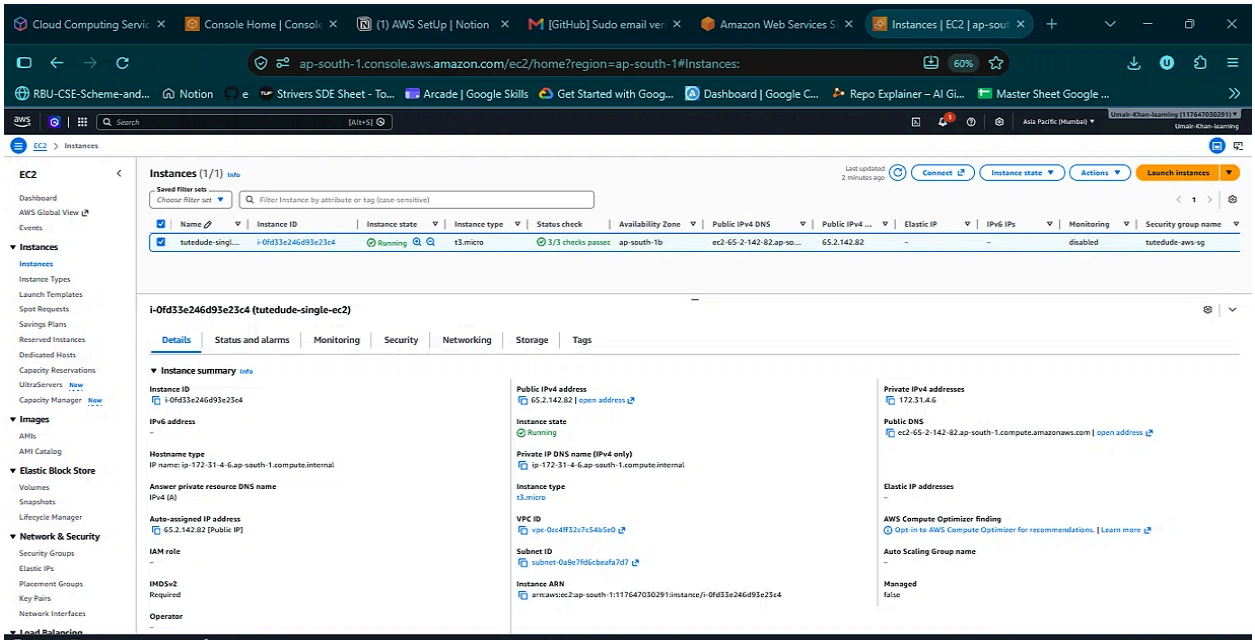


Figure 3: EC2 instance details showing ID, public IP, and security group.

3.5 Live URL

http://65.2.142.82:3000

Instance was stopped after verification to avoid charges, per assignment instructions

.

4. Task 2 — Deployment on Separate EC2 Instances

4.1 Architecture

Browser → Frontend EC2 (15.206.205.213:3000) → HTTP → Backend EC2 (13.203.228.83:5000)

4.2 Backend EC2

Field	Value
Instance Name	`tutede-backend-ec2`
Instance ID	`i-06345c81ec96b3d63`
Public IP	`13.203.228.83`
Security Group	`tutede-backend-sg`
Ports Open	22 (SSH), 5000 (Flask)

4.3 Frontend EC2

Field	Value
Instance Name	`tutede-frontend-ec2`

Instance ID	`i-0353be89a825857c6`
Public IP	`15.206.205.213`
Security Group	`tutede-frontend-sg`
Ports Open	22 (SSH), 3000 (Express)

4.4 Deployment Commands

Backend EC2:

bash

ssh -i tutedude-aws-key-mumbai.pem ubuntu@13.203.228.83

sudo apt install -y python3 python3-pip python3-venv git

git clone https://github.com/Umair-bey/tutedude-assignments.git

cd tutede-assignments/Assignment-6/backend

python3 -m venv venv && source venv/bin/activate

pip install -r requirements.txt

nohup python app.py > backend.log 2>&1 &

Frontend EC2:

bash

ssh -i tutedude-aws-key-mumbai.pem ubuntu@15.206.205.213

sudo apt install -y git curl

curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -

sudo apt install -y nodejs

git clone https://github.com/Umair-bey/tutedude-assignments.git

cd tutede-assignments/Assignment-6/frontend

npm install

BACKEND_URL=http://13.203.228.83:5000 nohup node app.js > frontend.log 2>&1 &

4.5 Troubleshooting Note

The frontend security group originally had **outbound rules restricted to SSH only**, which blocked `apt update` and `npm install` (they couldn't reach the internet). This was fixed by changing the outbound rules to **All traffic / 0.0.0.0/0**, allowing package installation to complete.

4.6 Screenshots

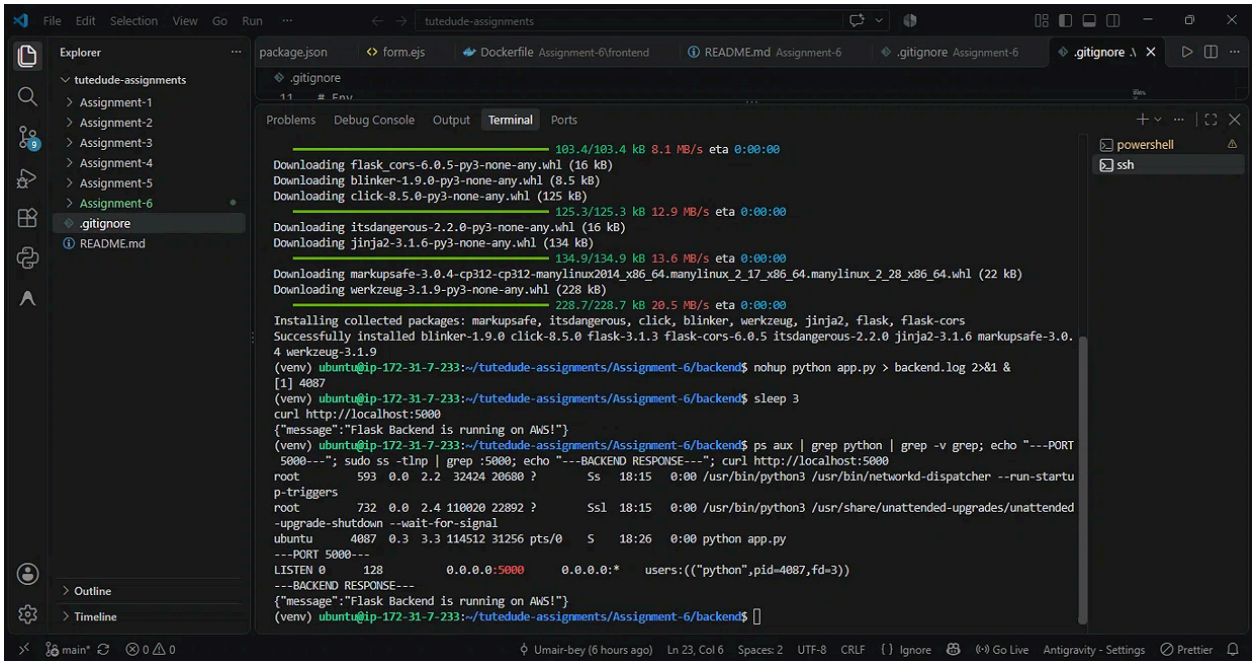


Figure 4: Flask backend running on the dedicated backend EC2.

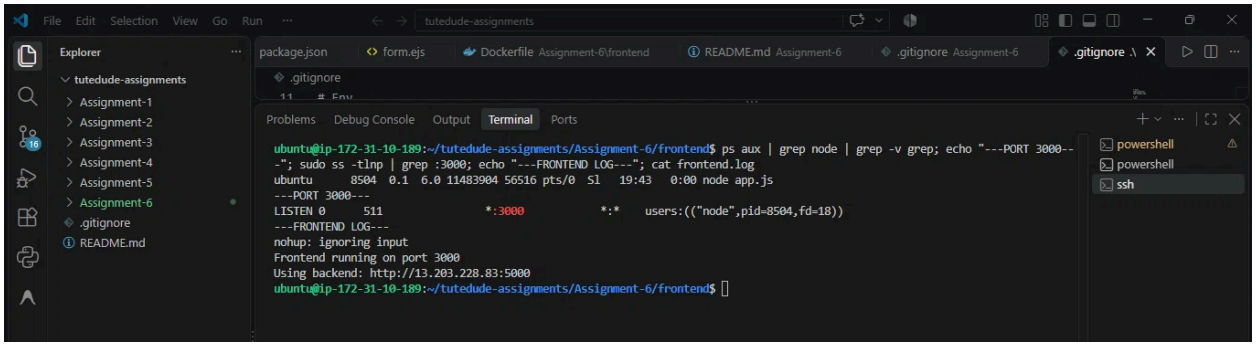


Figure 5: Express frontend running with BACKEND_URL pointing at the backend EC2.

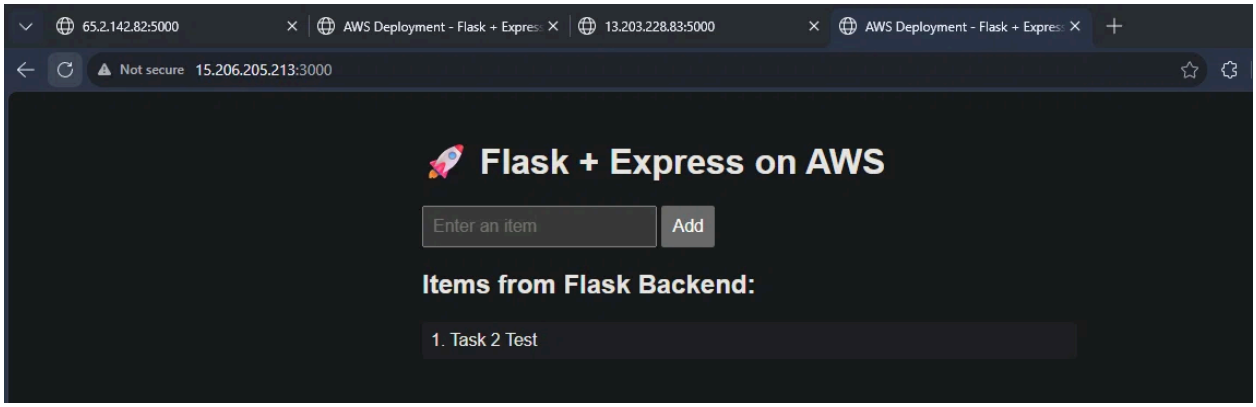


Figure 6: Two-server architecture working live at http://15.206.205.213:3000.

4.7 Live URL

http://15.206.205.213:3000

5. Task 3 — Docker + ECR + ECS + VPC

5.1 Architecture

Browser → ECS Frontend Task (13.201.96.116:3000)

→ HTTP → ECS Backend Task (43.204.231.169:5000)

Both containers run as **Fargate tasks** inside a custom VPC.

5.2 Docker Images

Both images built locally and pushed to **Amazon ECR**:

Image	ECR Repository	Size
`flask-backend:latest`	`117647030291.dkr.ecr.ap-south-1.amazonaws.com/flask-backend`	54.9 MB
`express-frontend:latest`	`117647030291.dkr.ecr.ap-south-1.amazonaws.com/express-frontend`	73.2 MB

Push commands:

bash

aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin 117647030291.dkr.ecr.ap-south-1.amazonaws.com

docker tag flask-backend:latest 117647030291.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest

docker push 117647030291.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest

5.3 Custom VPC

Resource	Value

VPC Name	`tutede-vpc-vpc`
VPC ID	`vpc-0f327aa819d1654a1`
IPv4 CIDR	10.0.0.0/16
Subnet 1	`tutede-vpc-subnet-public1-ap-south-1a`
Subnet 2	`tutede-vpc-subnet-public2-ap-south-1b`
Internet Gateway	`tutede-vpc-igw`
Route Table	`tutede-vpc-rtb-public` (0.0.0.0/0 → IGW)

5.4 Security Groups

SG Name	Port Open	VPC
`tutede-ecs-backend-sg`	5000	tutede-vpc-vpc
`tutede-ecs-frontend-sg`	3000	tutede-vpc-vpc

5.5 ECS Cluster & Task Definitions

Cluster: `tutede-cluster` (Fargate only)

Task Definition	Container	Image	Port	CPU/Mem
`tutede-backend-task:1`	flask-backend	flask-backend:latest	5000	0.25 vCPU / 0.5 GB
`tutede-frontend-task:2`	express-frontend	express-frontend:latest	3000	0.25 vCPU / 0.5 GB

The frontend task definition (revision 2) includes the environment variable:

BACKEND_URL = http://43.204.231.169:5000

5.6 ECS Services

Service	Task Definition	Desired Count	Public IP
`tutede-backend-service`	`tutede-backend-task:1`	1	43.204.231.169
`tutede-frontend-service`	`tutede-frontend-task:2`	1	13.201.96.116

Both services use:

- Launch type: Fargate
- Networking: VPC `tutede-vpc-vpc` with both public subnets
- Public IP: Enabled
- Security groups: ECS-specific

5.7 Screenshots

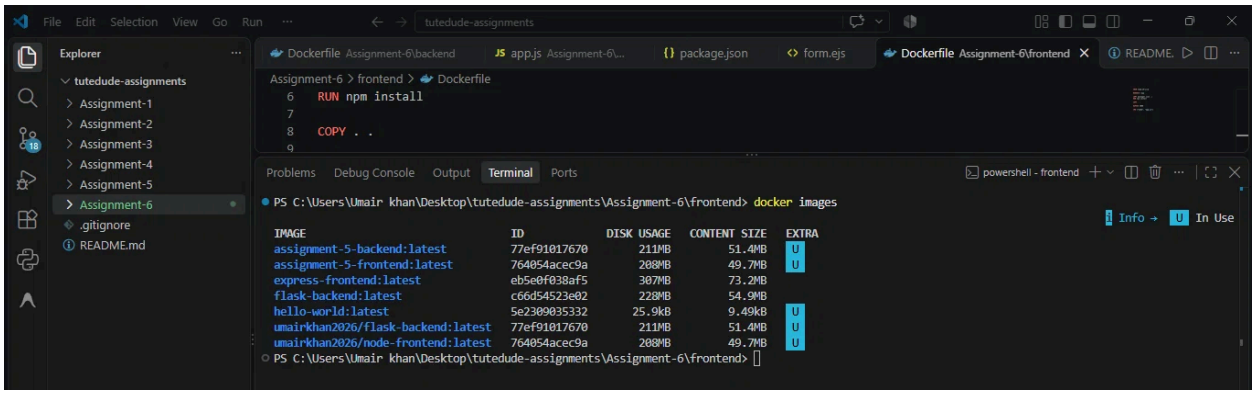


Figure 7: Both Docker images built locally (flask-backend and express-frontend).

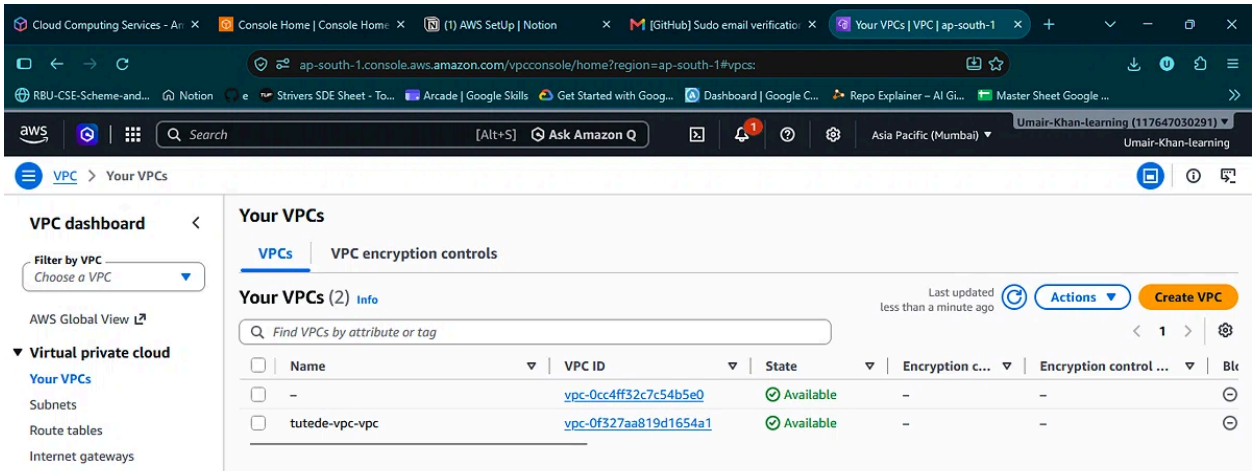


Figure 8: Custom VPC created with 2 public subnets and an Internet Gateway.

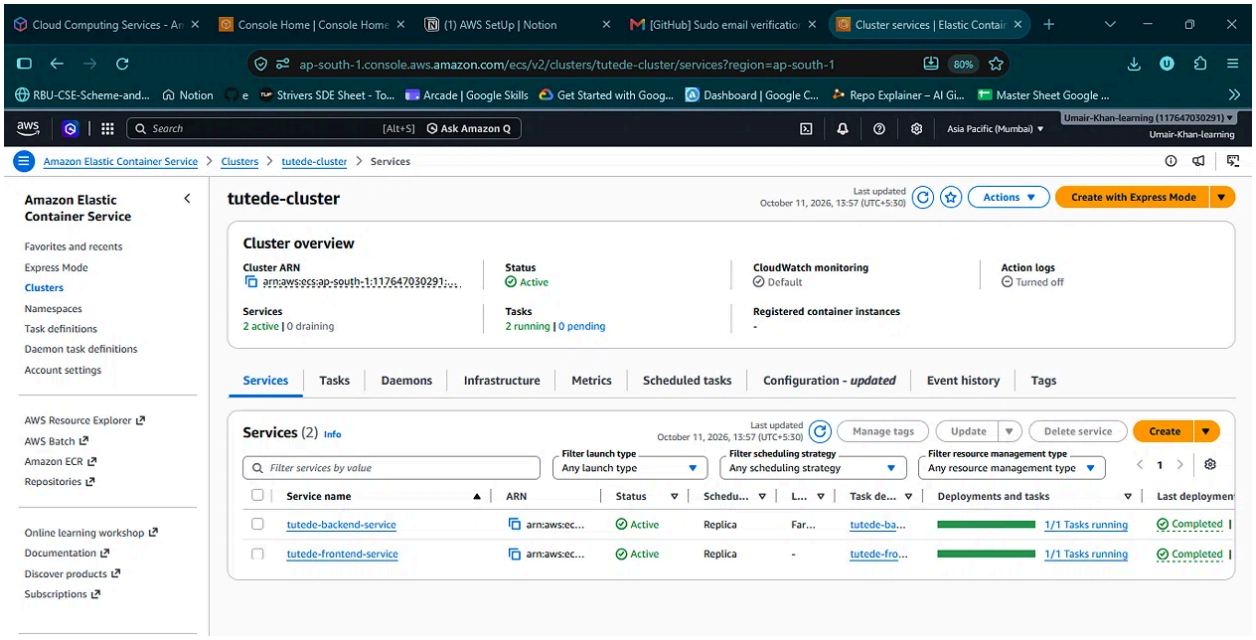


Figure 9: Both ECS services active — 2 running tasks.

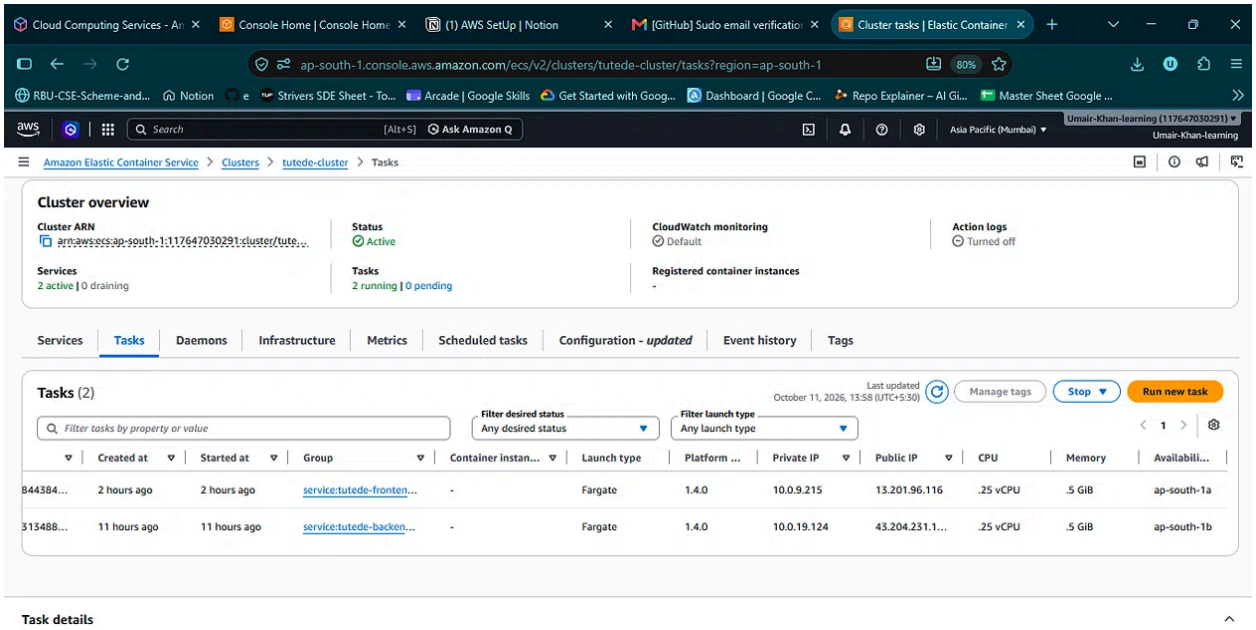


Figure 10: ECS task list showing public IPs for both containers.

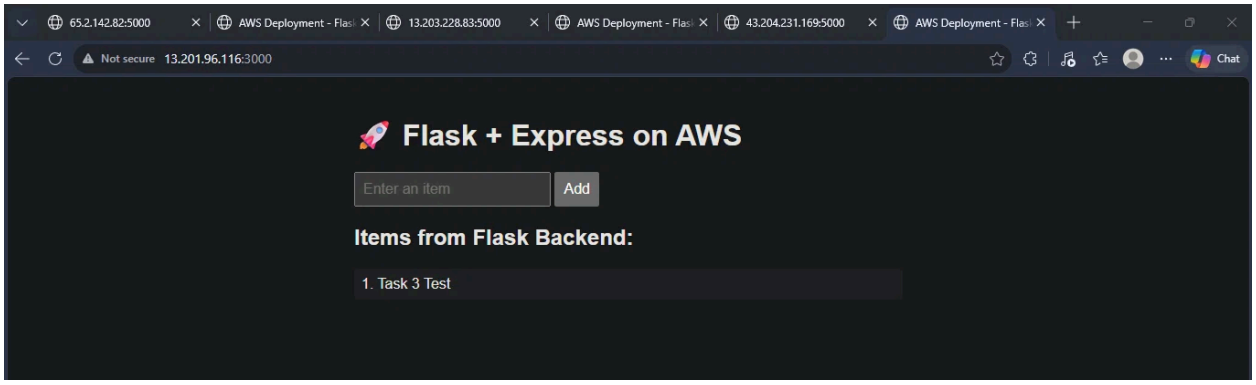


Figure 11: Full application live on ECS Fargate at http://13.201.96.116:3000.

5.8 Live URL

http://13.201.96.116:3000

5.9 Cleanup

After verification, both ECS services were updated to **Desired tasks = 0** to stop container charges while preserving the service and task definitions. EC2 instances from Tasks 1 and 2 were also stopped. All resources can be restarted if needed.

6. GitHub Repository

Link: https://github.com/Umair-bey/tutedude-assignments

Complete source code for all assignments, including:

- `Assignment-6/backend/` — Flask app + Dockerfile
- `Assignment-6/frontend/` — Express app + Dockerfile
- `Assignment-6/screenshots/` — All deployment screenshots

7. Summary & Learnings

Task	Deployment Pattern	Status
1	Single EC2 — monolith	✅ Complete
2	Two EC2s — distributed	✅ Complete
3	ECS Fargate — containerized	✅ Complete

Key learnings:

1. **EC2 provisioning** — Launching, securing, connecting to virtual servers
2. **Security Groups** — Inbound AND outbound rules matter; misconfigured egress blocks package installation
3. **Process management** — `nohup` keeps apps running after SSH disconnect
4. **Inter-server communication** — Frontend EC2 talking to backend EC2 via public IP
5. **Docker** — Building minimal images with layer caching
6. **ECR** — Private Docker registry with token-based authentication
7. **VPC** — Custom networking with public subnets, IGW, route tables
8. **ECS Fargate** — Serverless container orchestration with task definitions and services
9. **Cost awareness** — Stopping instances/services when not in use

8. Conclusion

All three deployment patterns were successfully implemented and verified. The application is fully functional in each architecture, demonstrating practical proficiency with core AWS services and DevOps workflows.